

Programmation orientée objet

Présentation : Tom Niget

Mail de contact : polytech@niget.fr

Site et ressources : niget.fr

Cours réalisé grâce notamment aux ressources de :
Julien Deantoni, Benjamin Werner, Joseph Albahari

Programmation orientée objet

- Paradigme de programmation : façon de programmer
 - **Programmation impérative** : on donne des ordres à la machine, la machine obéit
 - Assembleur, BASIC, C, Python, Java, C++, C#, JavaScript, ...
 - **Programmation déclarative** : on décrit un résultat à la machine, la machine se débrouille pour nous le donner
 - SQL, CSS, HTML, Prolog, ...
- (liste non exhaustive)

Programmation orientée objet

- **Programmation impérative** : on donne des ordres à la machine, la machine obéit
 - Assembleur, BASIC (Casio, TI, ...), ...
 - **Programmation procédurale** : on divise le programme en sous-programmes (procédures, aussi appelées fonctions) pour pouvoir réutiliser des morceaux de logique
 - C, Python, Java, C++, C#, JavaScript, ...

Programmation orientée objet

- **Programmation procédurale** : on divise le programme en sous-programmes (procédures, aussi appelées fonctions) pour pouvoir réutiliser des morceaux de logique
 - C, Pascal, Fortran, ...
 - **Programmation orientée objet** : on encapsule les données et les comportements des entités
 - Java, C++, C#, ...

Programmation orientée objet

```
float solde = 100;  
solde = solde + 50;  
solde = solde - 20;  
solde = solde - 10;  
solde = solde + 8;  
solde = solde * 1.01;
```

Impératif

- Code très concis
- Duplication de code
- Que « fait » chaque ligne ?

Encapsulation des comportements

Procédural

- Chaque comportement est isolé
- On voit quelle opération est faite grâce au nom de la fonction
- Rien n'empêche de modifier le solde à la main

```
void credit(float* solde, float montant) {  
    *solde += montant;  
}  
void debit(float* solde, float montant) {  
    *solde -= montant;  
}  
void interets(float* solde, float taux) {  
    credit(solde, *solde * taux);  
}  
bool est_riche(float solde) {  
    return solde > 1000000;  
}  
float solde = 100;  
float taux_interets = 0.01;  
credit(&solde, 50);  
debit(&solde, 20);  
debit(&solde, 10);  
credit(&solde, 8);  
interets(&solde, taux_interets);  
printf(est_riche(solde) ? "Riche" : "Pas riche");
```

Programmation orientée objet

```
void credit(float* solde, float montant) {
    *solde += montant;
}
void debit(float* solde, float montant) {
    *solde -= montant;
}
void interets(float* solde, float taux) {
    credit(solde, *solde * taux);
}
bool est_riche(float solde) {
    return solde > 1000000;
}
float solde = 100;
float taux_interets = 0.01;
credit(&solde, 50);
debit(&solde, 20);
debit(&solde, 10);
credit(&solde, 8);
interets(&solde, taux_interets);
printf(est_riche(solde) ? "Riche" : "Pas riche");
```

```
void debit(float* solde, float montant) {
    if (montant > solde) {
        throw BankException("Solde insuffisant");
    }
    solde -= montant;
}
```

```
float solde = 100;
debit(&solde, 9000); // Exception: Solde insuffisant
solde -= 9000; // Pas d'exception, solde négatif
```

Programmation orientée objet

Données
(variables)

```
class CompteBancaire {  
    float solde;
```

```
public:
```

```
    CompteBancaire(float solde_initial) : solde(solde_initial) {}
```

Comportements
(fonctions / méthodes)

```
    void credit(float montant) { this->solde += montant; }  
    void debit(float montant) {  
        if (montant > this->solde) throw BankException("Solde insuffisant");  
        this->solde -= montant;  
    }  
    void interets(float taux) { credit(this->solde * taux); }  
    bool est_riche() const { return this->solde > 1000000; }  
    float get_solde() const { return this->solde; }  
};
```

`private` (par défaut) = inaccessible
depuis l'extérieur

`public` = accessible par l'extérieur

`this` sert à accéder aux membres de la classe

Programmation orientée objet

- On décrit les entités comme des **classes**
- Les classes **encapsulent** les **données** et les **comportements**
- L'encapsulation permet de préserver des **invariants**, des contraintes, et donc aide à garantir la cohérence du programme

Programmation orientée objet

(essentiellement un cours de C++)

- En C, on peut prendre l'adresse d'une variable : on obtient un pointeur
 - + : on peut passer indirectement une valeur et permettre à une fonction de la modifier ; on peut faire référence à une valeur sans la copier
 - - : syntaxe spéciale (->), possibilité de pointeur nul ou invalide
- En C++, il y a un deuxième moyen de faire ça : les références (et oui...)
 - + : pas de syntaxe spéciale (une référence se comporte comme un « alias » pour la valeur référencée) ; une référence ne peut pas être nulle
 - - : on ne peut pas changer vers quoi pointe une référence après la création

Programmation orientée objet

(essentiellement un cours de C++)

```
int x = 5;  
int* ptr_x = &x;  
*ptr_x = 123; // nécessité de *
```

```
struct s { int t };  
struct s val_s;  
struct s* ptr_s = val_s;  
ptr_s->t = 123; // nécessité de ->
```

```
int x = 5;  
int& ref_x = x;  
ref_x = 123; // pas de *
```

```
struct s { int t };  
s val_s;  
s& ref_s = val_s;  
ref_s.t = 123; // pas de ->
```

Autre différence par rapport au C : pas besoin de préciser « `struct` » pour les types structures, pas besoin de `typedef`

Programmation orientée objet

(essentiellement un cours de C++)

```
int* fonction_bizarre() {  
    int mavaleur = 5;  
    int* ptr = &mavaleur;  
    return ptr;  
}
```

```
int& fonction_bizarre() {  
    int mavaleur = 5;  
    int& ref = mavaleur;  
    return ref;  
}
```

Le C ne vérifie aucun pointeur.
Le C++ ne permet pas de créer de référence nulle, mais ne vérifie pas que les références restent valides.

Vous aurez peut-être des warnings, mais c'est tout !

Programmation orientée objet

(essentiellement un cours de C++)

- Tout objet (valeur, peu importe son type) a une **durée de vie**
- La durée de vie d'un objet dépend de là où il est stocké :
 - Variable **globale** (en dehors d'une fonction) ou **statique** : du début à la fin du programme
 - Variable **locale** (dans un bloc { }) : cesse d'exister à la fin du bloc
 - Allocation **dynamique** (`malloc`, `new`) : de l'allocation, jusqu'à la désallocation (ou la fin du programme)
- Un pointeur ou une référence vers un objet n'est valide que tant que l'objet cible **est en vie**
- Utiliser un pointeur/référence invalide est **illégal** (et peut provoquer à peu près n'importe quoi), mais votre code compilera quand même !

Programmation orientée objet

(essentiellement un cours de C++)

```
int main() {  
    T o1 = T(123); // construction de l'objet o1  
  
    {  
        T o2 = T(456); // construction de l'objet o2  
        ...  
    } // fin du bloc : o2 est détruit  
  
    ...  
    return 0;  
} // fin du bloc : o1 est détruit
```

Programmation orientée objet

(essentiellement un cours de C++)

- En C++, la construction d'un objet appelle le constructeur, et sa destruction appelle son destructeur.
- Pour un type **T**, le constructeur est noté **T(. . .)** et le destructeur **~T(. . .)**
- Le constructeur peut prendre des paramètres
- Par défaut, le constructeur et le destructeur sont vides (ne font rien de supplémentaire)

Programmation orientée objet

(essentiellement un cours de C++)

- En C++, la construction d'un objet appelle le constructeur, et sa destruction appelle son destructeur.
- Pour un type **T**, le constructeur est noté **T(. . .)** et le destructeur **~T(. . .)**
- Le constructeur peut prendre des paramètres
- Par défaut, le constructeur et le destructeur sont vides (ne font rien de supplémentaire)

Programmation orientée objet

(essentiellement un cours de C++)

- C++ a des mots-clés qui correspondent à `malloc/free` : `new` et `delete`
- Avantages par rapport au C : ils sont typés (renvoient un `T*`) et connaissent automatiquement la taille

```
struct T* obj = malloc(sizeof(T));  
obj->foo = 123;  
free(obj);
```

```
T* obj = new T();  
obj->foo = 123;  
delete obj;
```

- Mais même défaut qu'en C : il faut se rappeler de libérer quand on n'a plus besoin de l'objet, sinon il reste alloué tout le long du programme

Programmation orientée objet

(essentiellement un cours de C++)

- Concept très pratique de C++ : le RAII
 - « Resource Acquisition Is Initialization »... ça ne veut pas dire grand chose
- L'idée : des types utilitaires qui gèrent automatiquement l'allocation et la libération de ressources (comme de la mémoire, des fichiers, ...)

```
{  
    std::unique_ptr<T> obj = std::make_unique<T>();  
    obj->foo = 123;  
} // le unique_ptr est détruit et libère la mémoire
```